CHAPTER 1

DESIGNING A JIGSAW PUZZLE

1 Getting Ready: Jigsaw Puzzle 2
2 Requirements for the Jigsaw Puzzle . 3
3 Class Diagram for the Jigsaw Puzzle 3
4 Code of Jigsaw Puzzle 8

In this chapter, you will discover the design of a jigsaw puzzle, focusing on requirements, class diagram, and implementation code for the puzzle.

1 Getting Ready: Jigsaw Puzzle

1.1 Problem definition

A **jigsaw puzzle** is a picture divided into irregularly shaped pieces that can be assembled to form a complete picture. It has a fixed number of pieces that can only fit together in exactly one way. We can solve the jigsaw puzzle by aligning the pieces to fit together and create a coherent picture.

1.2 Expectations from the interviewee

Although the jigsaw problem is a simpler design problem asked in interviews, the interviewer still has some expectations. The following provides an overview of what the interviewer wants to hear you discuss in more detail during the interview.

The puzzle board

You may want to ask the interviewer more about the kind of jigsaw puzzle. You can ask the following questions:

- What does a jigsaw puzzle look like? Is there a picture or a pattern on the puzzle?
- What type of shape does our board have? Is it rectangular, circular, or any other shape?

The puzzle pieces

It is important to clarify the requirements for a single piece of the puzzle, as it is a key feature of the problem. You can ask the following questions:

- What kind of shape do the puzzle pieces have? How many sides does a piece have?
- How do the pieces fit together?
- Will all the pieces of a jigsaw puzzle be unique?

1.3 Design approach

We are going to design this jigsaw puzzle using the bottom-up design approach. For this, we will follow the steps below:

- Identify and design the smallest components first, like the puzzle edges of a piece.
- Use these small components to design bigger components, for example, the puzzle piece.
- Repeat the steps above until we design the whole system, i.e., the puzzle board.

1.4 Design pattern

During an interview, discussing the design patterns that a Jigsaw puzzle falls under is always a good practice. Stating the design patterns gives the interviewer a positive impression and shows that the interviewee is well-versed in the advanced concepts of object-oriented design.
Try to answer the following question. If you are not familiar with design patterns, don't worry! You can learn about them by asking, "Define design patterns."

Question:
Which design pattern(s) should be used to design a jigsaw puzzle? Please elaborate on your choice(s).

> **Note:** The jigsaw puzzle design problem is fairly simple, so we don't need to create the use case, sequence, and activity diagrams. The requirements defined for this problem are unsuitable for these diagrams.

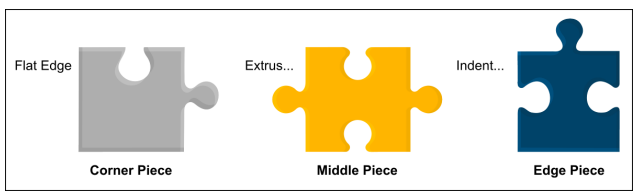
Let's explore the requirements of the jigsaw puzzle problem in the next lesson.

2 Requirements for the Jigsaw Puzzle

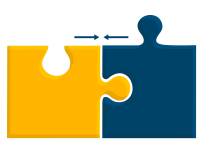
In this lesson, we'll list the requirements of our jigsaw puzzle problem. This is a crucial step since requirements define the scope of a problem. Getting them right from the interviewer and understanding them well will make the design of the rest of the problem smooth and easy.
We'll use the notational convention to identify each requirement with a unique label "Rn," where "R" is short for Requirement and "n" is a natural number.

2.1 Requirement collection

For the jigsaw puzzle problem, the requirements are defined below:
R1: Our board will be in the shape of a rectangle.
R2: All pieces will have four sides with an indentation, an extrusion, or a flat edge.
R3: There are four corner pieces, some edge pieces, and the remaining ones are the middle pieces. A corner piece has two flat sides, an edge piece only has one flat side, and a middle piece has no flat edges.



R4: All pieces will be unique, so only one piece will fit with one other piece.
R5: Two pieces fit together by the curvature of the indentation on one piece matching up to the curvature of the extrusion on another.



We've identified our requirements for the problem, and in the next lesson, we will define the class diagram of our jigsaw puzzle system.

3 Class Diagram for the Jigsaw Puzzle

In this lesson, we'll identify and design the classes, abstract classes, and interfaces based on the requirements we previously gathered from the interviewer in our jigsaw puzzle.

3.1 Components of a jigsaw puzzle

As mentioned, we should design the jigsaw puzzle using a bottom-up approach.

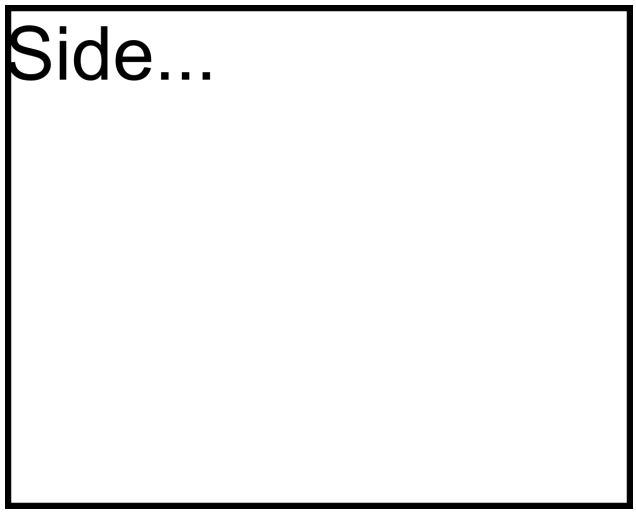


Figure 1.1: The class diagram of the Side class

Side

The `Side` class represents the shape of our jigsaw piece and whether it contains an indentation, extrusion, or flat edge. The UML representation of the class is shown below:

R2: All pieces will have four sides with an indentation, an extrusion, or a flat edge.

Piece

The `Piece` class contains an array of sides of size four. It will also identify middle, corner, and edge pieces. The class representation of the `Piece` class is provided below:

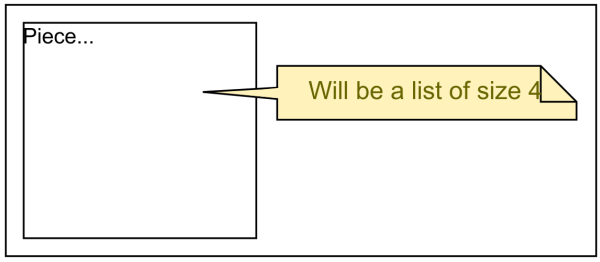


Figure 1.2: The class diagram of the Piece class

R2: All pieces will have four sides with an indentation, an extrusion, or a flat edge.
R3: There are four corner pieces, some edge pieces, and the remaining ones are middle pieces. A corner piece has two flat sides, an edge piece only has one flat side, and a middle piece has no flat edges.

Puzzle

The `Puzzle` class represents the board of our jigsaw game. As our board is a rectangle, it will be represented by a 2D array. There will also be a 1D array to represent the unused free pieces yet to be inserted into the puzzle board. It will also have the `insertPiece()` function to insert the piece into the board, ensuring that it is unique from its counterparts and only in the specified row and column.

> **Note:** The puzzle board does not yet have the functionality of rotating pieces, so all pieces must be unique to fit into the board and solve the puzzle.

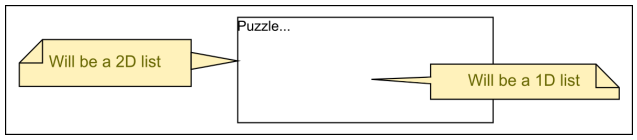


Figure 1.3: The class diagram of the Puzzle class

This class is represented below:

R1: Our board will be shaped like a rectangle.

R4: All pieces will be unique, so only one piece will fit with only one other piece.

Puzzle solver

The `PuzzleSolver` class is responsible for solving an unsolved jigsaw puzzle board using its `matchPieces()` function. The visual representation of this class is given below:

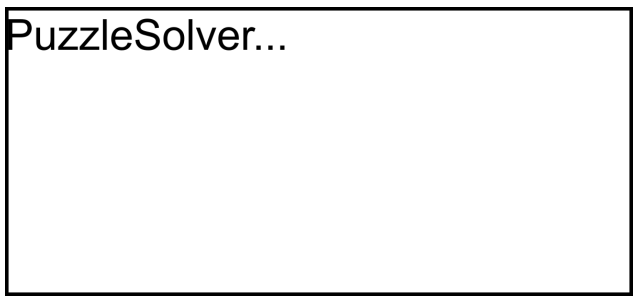


Figure 1.4: The class diagram of the PuzzleSolver class

R5: Two pieces fit together by the curvature of the indentation on one piece matching up to the curvature of the extrusion on another.

Edge enumeration

The `Edge` enum describes the various edges present in a jigsaw puzzle piece. It is represented using the class diagram given below:

3.2 Relationship between the classes

Now, we'll discuss the relationships between the classes we have defined above in our jigsaw puzzle.

Association

The class diagram has the following association relationships:

- The `Puzzle` has a one-way association with `PuzzleSolver`.

Composition

The class diagram has the following composition relationships:

- The `Puzzle` class is composed of the `Piece` class, which is composed of the `Side` class.

3.3 Class diagram for the jigsaw puzzle

Here's the complete class diagram for our jigsaw puzzle:

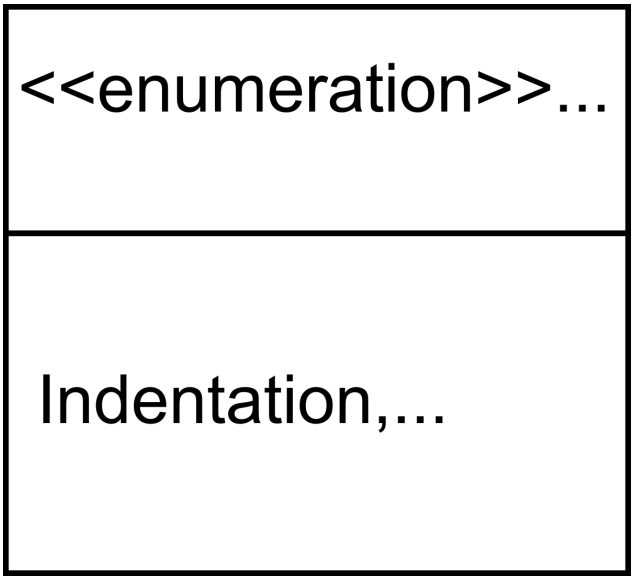


Figure 1.5: The class diagram of the Edge enum

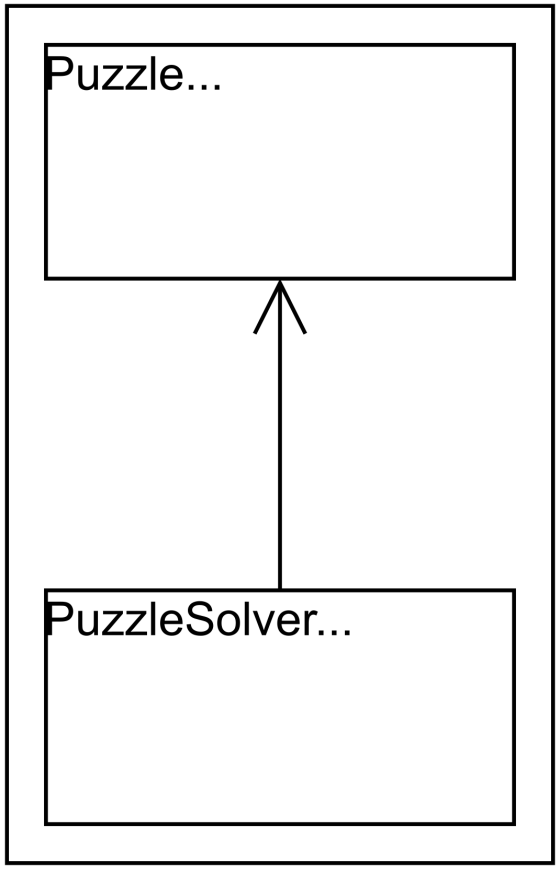


Figure 1.6: The association relationship between classes

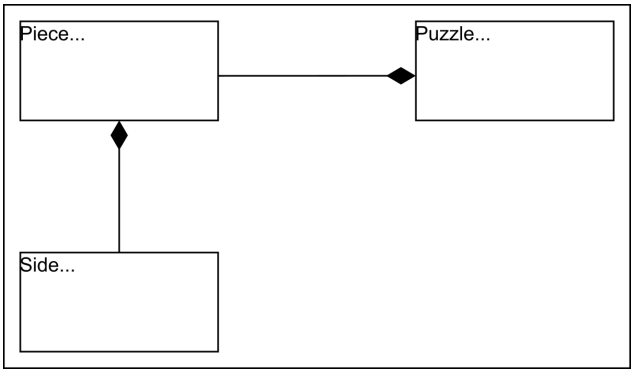


Figure 1.7: The composition relationship between classes

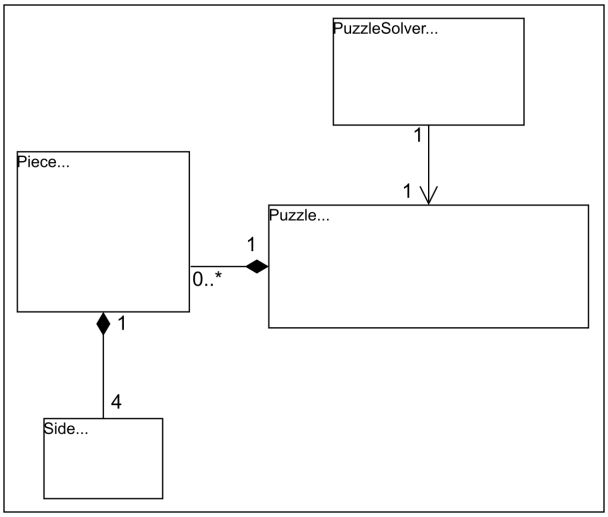


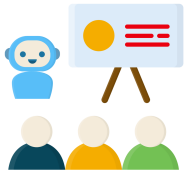
Figure 1.8: The class diagram of the jigsaw puzzle

3.4 Design pattern

The jigsaw puzzle has only one instance of the puzzle board. Therefore, we use the Singleton design pattern to ensure that only one instance of the board is created using a special creation method, and this instance has a global point of access.

3.5 AI-powered trainer

At this stage, everything should be clear. If you encounter any confusion or ambiguity, feel free to utilize the interactive AI-enabled widget below to seek clarification. This tool is designed to help you strengthen your understanding of the concepts.



3.6 Additional requirements

The interviewer can introduce some additional requirements in the jigsaw puzzle, or they can ask some follow-up questions. Let's see some examples of additional requirements:
Rotate piece: Pieces can be rotated to fit on the puzzle. The class diagram provided below shows the added functionality of rotation in the `Piece` class:

We have completed the class diagram of the jigsaw puzzle according to the requirements. Let's write the code for our designed classes in some popular languages.

4 Code of Jigsaw Puzzle

We've reviewed the different aspects of the jigsaw puzzle and observed the attributes attached to the problem using various UML diagrams. Now , let's explore the more practical side of things, where we will work on implementing the jigsaw puzzle using multiple languages. This is usually the last step in an object-oriented design interview process.
We have chosen the following languages to write the skeleton code of the different classes present in the jigsaw puzzle:

- Java
- C#
- Python
- C++
- JavaScript

4.1 Jigsaw puzzle

This section will provide the skeleton code of the classes designed in the class diagram lesson.

> **Note:** For simplicity, we are not defining getter and setter functions. The reader can assume that all class attributes are private, accessed through their respective public getter methods, and modified only through their public method functions.

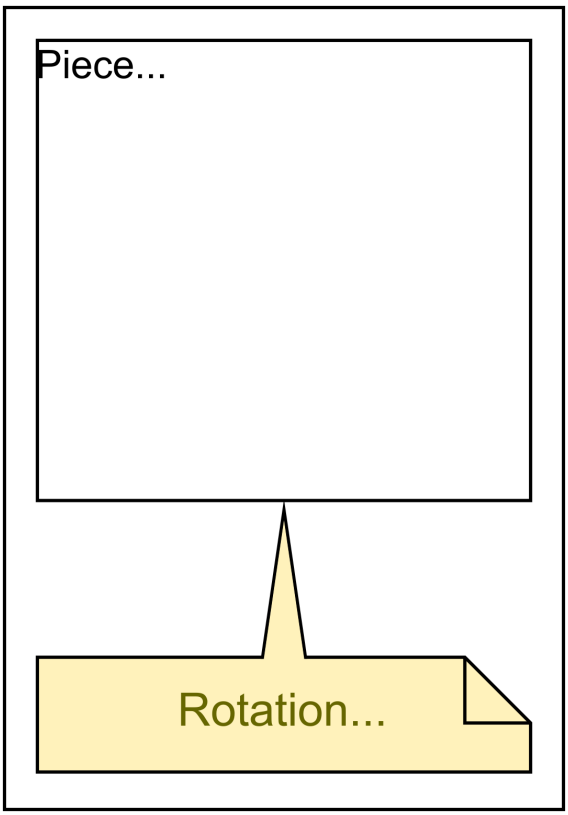


Figure 1.9: The class diagram of the rotation functionality in the Piece class

Enumerations

The following code defines the `Edge` enum that represents the shape of the puzzle piece:

> **Note:** JavaScript does not support enumerations, so we will use the `Object.freeze()` method as an alternative that freezes an object and prevents further modifications.

Enum definitions

```
enum Edge {
  INDENTATION,
  EXTRUSION,
  FLAT
}
```

Side

The `Side` class contains a reference to the `Edge` enum that will describe the edge shape of any side of a puzzle piece. Its definition is given below:

The Side class

```
public class Side {
  private Edge edge;
}
```

Piece

The `Piece` class contains a list of sides (representing each side of a piece) and ensures that a particular piece is a corner, middle, or edge using its respective functions. The definition of this class is provided below:

The Piece class

```
public class Piece {
  private List<Side> sides = Arrays.asList(new Side[4]);
  public boolean checkCorner() {}
  public boolean checkEdge() {}
  public boolean checkMiddle() {}
}
```

Puzzle

The `Puzzle` class includes the entire board of the jigsaw puzzle and the unused pieces. As the board is a rectangle, it will be represented using a 2D array. In addition, there will be only one instance of the puzzle board in the jigsaw puzzle. Therefore, the `Puzzle` class will be a Singleton class to ensure that only one instance of the board is created in the entire system. The definition of this class is given below:

The Puzzle class

```
public class Puzzle {
  private List<List<Piece>> board;
  private List<Piece> free; // represents the currently free pieces (yet to be inserted in board)
  public void insertPiece(Piece piece, int row, int column) {};
```

```
  // Puzzle is a singleton class that ensures it will have only one active instance at a time
  private static Puzzle puzzle = null;

  // Created a static method to access the singleton instance of Puzzle class
  public static Puzzle getInstance() {
      if (puzzle == null) {
          puzzle = new Puzzle();
      }
      return puzzle;
  }
}
```

Puzzle solver

The `PuzzleSolver` class tries to solve the puzzle using the `matchPieces()` function. Its definition is given below:

The PuzzleSolver class

```
public class PuzzleSolver {
  public Puzzle matchPieces(Puzzle board) {}
}
```

4.2 Wrapping up

In this chapter, we've explored the complete design of a jigsaw puzzle. We 've looked at how a basic jigsaw puzzle can be visualized using various UML diagrams and designed it using object-oriented principles and design patterns.